

Chapter 6 – Multiple Parameters and Arguments

SLIDE 1 — What Are Multiple Parameters?

Definition

Multiple parameters are two or more variables used in a function to receive different values.

They allow one function to work with more than one input.

Analogy

Think of a shopping bill:

- **price** → Cost of one item
- **quantity** → Number of items
- The function uses both values to calculate the total.

Multiple inputs → One task → One result

Small Example

```
None
void bill(int price, int quantity) {
    printf("%d", price * quantity);
}
```

Here, **price** and **quantity** are two parameters.

SLIDE 2 — Why Are Multiple Parameters Used?

Why do we use them?

Multiple parameters are used to:

1. Pass more than one value to a function.
2. Perform calculations using different inputs.

3. Make functions flexible and reusable.
4. Avoid creating separate functions for similar tasks.
5. Improve program organization.
6. Represent real-world problems more clearly.

Important Statement ★

Each parameter receives a value from the corresponding argument.

Parameter Order

The order of arguments is important.

None

```
add(5, 3);
```

- - 5 goes to the first parameter.
- 3 goes to the second parameter.

Changing the order may change the result.

SLIDE 3 — Parameters vs Arguments

Parameters 🧠

Parameters are variables written inside the function definition.

None

```
void add(int a, int b)
```

Here, **a** and **b** are parameters.

Arguments 🧠

Arguments are the actual values passed during the function call.

None

```
add(5, 3);
```

Here, 5 and 3 are arguments.

Comparison ★

Parameters	Arguments
Written in function definition	Written in function call
Receive values	Provide values
Act like variables	Act like actual data

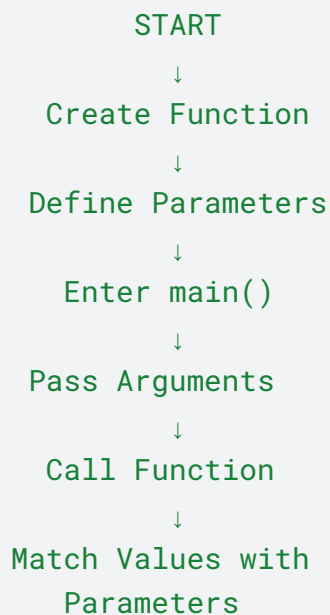
Important Statement 📌

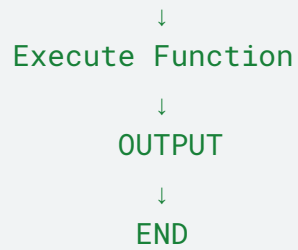
Parameters receive; arguments provide.

SLIDE 4 — Flowchart and Function Execution

Flowchart

None





Example

None

```
#include <stdio.h>

void add(int a, int b) {
    printf("%d", a + b);
}

int main() {
    add(7, 2);
    return 0;
}
```

Output

None

9

Explanation

- `a` receives 7.
- `b` receives 2.
- The function adds both values.
- The result 9 is displayed.

SLIDE 5 — Recap

Quick Recap

- Multiple parameters allow a function to receive two or more values.
- Parameters are written in the function definition.
- Arguments are the actual values passed during the function call.
- Each argument is matched with its corresponding parameter.
- The order of arguments is important.
- Multiple parameters make functions reusable and flexible.

Example

None

```
void add(int a, int b) {  
    printf("%d", a + b);  
}
```

None

```
add(5, 3);
```

Output:

None

8

Memory Trick

Parameters receive. Arguments provide

Chapter 6 – Multiple Parameters and Arguments

Practice Questions

1. Theory MCQ

What is the purpose of using multiple parameters in a function?

A. To accept only one value B. To accept two or more values C. To stop the function D. To print only text

💡 Hint: Parameters 📖 Glossary: Parameter — A value received by a function.

Answer: B. To accept two or more values

2. Output Guess

None

```
#include <stdio.h>

void add(int a, int b) {
    printf("%d", a + b);
}

int main() {
    add(6, 4);
    return 0;
}
```

What will be the output?

💡 Hint: Addition 📖 Glossary: Argument — The actual value passed to a function.

Answer: 10

3. Fill in the Missing Code

None

```
#include <stdio.h>

void student(char name[], _____ age) {
    printf("%s %d", name, age);
}

int main() {
    student("Arun", 18);
    return 0;
}
```

💡 Hint: Integer 📖 Glossary: Data type — Specifies the type of value stored.

Answer: `int`

4. Jumbled Sentence

Arrange the words in the correct order:

value / A / return / function / can / a

💡 Hint: Return 📖 Glossary: Return — Sends a value back from a function.

Answer: A function can return a value.

5. Fill in the Missing Code

None

```
#include <stdio.h>

int _____() {
    return 25;
}

int main() {
    printf("%d", getNumber());
    return 0;
}
```

💡 Hint: Name 📖 Glossary: Function name — The name used to call a function.

Answer: `getNumber`